# 민예린

Fullstack Developer · Spring Boot · FastAPI · Next.js · AI

**이메일** myelin24@naver.com
**출생** 1994
**학력** 한성대학교 행정학과 졸업 (경제학 부전공)
**자격** GAIQ (2021) · MOS Master (2015)
**희망연봉** 회사 내규에 따름

현실 데이터를 현장이 보는 화면으로 만들고, AI와 웹 서비스로 연결하는 풀스택 개발자 민예린입니다.
회사에서는 운영과 B2B 웹을, 개인 프로젝트에서는 SDUI 엔진·비동기 큐·RAG를 만들었습니다. AI 도구는 구현 초안과 디버깅 후보를 만드는 데 쓰고, 설계 결정과 검증은 제가 책임집니다.

GitHub https://github.com/feed-mina 포트폴리오 https://feed-mina.github.io/
제조 데이터 · AI 자동화 사례 https://feed-mina.github.io/manufacturing-ai/
가이드롤 케이스 스터디 https://feed-mina.github.io/evol-case-study.html KMovement 라이브 https://yerin.duckdns.org/
work-cycle 공개 데모 https://work-cycle-portfolio.evolvix.workers.dev/
Template Kit Studio https://sdui-template-kit-productization.pages.dev/studio/

## 핵심

### 데이터는 기준부터: 제조 센서 지표는 정의를 먼저 맞춥니다

가이드롤 생산 모니터링에서 가동 횟수와 가동 시간이 다른 기준으로 계산돼 '가동 0회인데 1.2시간'이 뜨는 모순을 찾았고, 재합의안을 올려 회의에서 가동 기준을 확정하고 합산 함수를 하나로 모은 뒤 36일치 화면을 '항상 맞아야 할 규칙'으로 재계산해 대조하고 계약 테스트 26건으로 고정했습니다. 센서가 움직인 시간과 실제 생산 가동이 다른 지표라는 점은 표로 정리하고, 현장 확인이 필요한 항목은 미확정으로 남겼습니다.

### AI는 근거부터: 사람 승인을 거치는 흐름 안에 둡니다

AI 기획 루프(Planning Harness)와 회의록 요약 API(work-cycle)에서 밖으로 나가는 동작은 사람이 승인해야 실행되고, 검사에 걸린 결과는 버려 실패해도 흔적을 남기지 않게 했습니다. 임계값·정확 조건 필터·테스트·근거 기록으로 AI 결과를 검증 가능하게 두는 것을 중요하게 생각합니다.

### 운영 문제를 숫자와 기록으로 추적합니다

EBS 운영에서 모니터링 도구와 배치 집계의 수치가 어긋난 원인을 수집 주기·실패 처리·시간대 기준으로 좁혀 보강했고, 포스코DX 파견에서는 스팸 필터 변경을 테스트 서버에서 먼저 돌려 오탐을 확인하고 DMARC·SPF·DKIM 리포트를 주기적으로 점검했습니다.

### 화면과 기획을 코드가 아니라 데이터로 다룹니다

SDUI 엔진(DB의 화면 메타데이터를 Spring Boot가 트리로 바꾸고 Next.js가 재귀 렌더링) → Template Kit Studio 편집기(비개발자도 브라우저에서 수정) → Planning Harness 기획 루프까지 같은 규칙으로 이었고, KMovement에서는 FastAPI·ChromaDB·Celery/Redis 비동기 큐로 검색과 미디어 생성을 분리했습니다.

## 기술

| | |
|---|---|
| **Backend** | Java · Spring Boot · Spring Security · JSP · Python · FastAPI · C# / ASP.NET |
| **Data** | PostgreSQL · MySQL · Oracle · MSSQL · DuckDB · Cloudflare D1 · Redis · Parquet |
| **Manufacturing Data** | 센서 사이클·이벤트 JSONL · 시계열 집계 · 지표 정의표 · 불변식 테스트 |
| **AI** | RAG (ChromaDB · GraphRAG/Neo4j) · 임베딩 (multilingual-e5) · LangChain · LangGraph · LLM API (Gemini · OpenAI · Claude · Workers AI) · 승인 게이트 Agent 흐름 · Claude Code 스킬 (AI 협업) |
| **Frontend** | React · Next.js · Vue.js · JSP · jQuery · TypeScript · Streamlit |
| **Infra** | Docker · Jenkins · GitHub Actions · Cloudflare Workers · GCP · AWS · Linux Shell |
| **Verification** | pytest · Vitest · 계약 테스트 · 회귀 테스트 · 원본 데이터 대조 기록 |

## 경력 (개발 2년 6개월 · 기획 11개월)

2026.08 ~ 2026.09

### 이볼빅스 · 전임연구원 · 웹 개발

센서 데이터 조회·분석 웹 (FastAPI · React/TypeScript · DuckDB · Parquet · Docker · Jenkins)

- 가이드롤 생산 라인의 센서 노드(속도·진동) 측정값을 조회·분석하는 화면과 API, DB 구조를 구현하고 배포 작업을 수행했습니다.
- 통계 화면에서 가동 시간과 가동 횟수가 서로 맞지 않는 문제를 찾아 원인(세 곳이 다른 가동 기준 사용)을 정리하고, 재합의안을 올려 9월 14일 회의에서 가동 4조건을 확정했습니다. 합산 함수를 한 곳으로 통일한 뒤 36일치 화면을 '항상 맞아야 할 규칙'으로 재계산해 대조(위반 0건)하고 계약 테스트 26건으로 고정했습니다.
- 센서가 움직인 시간과 실제 생산 가동이 다른 지표라는 점을 지표 정의표로 정리하고, 현장 확인이 필요한 항목(속도 환산식·노드-롤 매핑·진동 임계값)은 미확정으로 남겨 두었습니다.

2025.08 ~ 2025.12

### 에이아이피플스 (포스코DX 파견) · 사원 · 웹 개발·운영

전사 스팸메일 차단 필터 운영 · DMARC 리포트 통계 (Oracle)

- 포스코DX 전사 스팸메일 차단 필터 시스템을 운영했습니다. 변경된 차단 로직은 테스트 서버에서 먼저 실행해 오탐을 확인한 뒤 반영했습니다.
- DMARC·SPF·DKIM 리포트를 모니터링하며 발신 도메인 신뢰도 통계를 주기적으로 확인했습니다.

2024.05 ~ 2025.02

### 유인시스 · 상록에스 (EBS 파견) · 계약직 사원 · 개발 운영

EBS 영어 교육 서비스 운영·통계 (Spring Boot · MySQL · Crontab · Whatap · Shell · Redis · Linux)

- Crontab으로 트래픽 API 호출과 CSV 저장을 자동화해 일일 보고 수작업을 줄였고, Whatap으로 주간 트렌드를 보며 이상 징후를 확인했습니다.
- Whatap과 Crontab 집계 수치가 어긋나는 것을 발견해 수집 주기·실패 대응 로직·시간대 기준을 통일했습니다.
- 운영·테스트 DB 100여 개 테이블의 구조와 약어를 정리한 데이터 딕셔너리를 만들었습니다.
- EBS 영어 도메인 분석 문서와 개인화 추천 로직(Xtraxtor) 분석 문서를 작성했습니다.

2023.06 ~ 2024.02

### 솔앤드 · 대리 · SI 개발

B2B 관리자 페이지·인증 (Spring Boot · Spring Security · JWT · OAuth2 · Vue.js · JSP · jQuery · MySQL · AWS S3)

- 순천향대 공자학당 비대면 학습 관리자 시스템을 Spring Boot + Vue2로 구축했습니다. 교수·조교·학생 역할별 권한을 분리하고, 웹에서 올린 갤러리를 앱에서 조회하는 연동 기능을 개발했습니다.
- JWT 인증과 OAuth2(카카오) 로그인을 구현하고, Axios 인터셉터로 토큰을 자동 포함시켜 인증 흐름을 단순화했습니다.
- KT 멤버십 이벤트 페이지를 S3 정적 호스팅으로 올리고 모바일·PC별 CSP를 적용했습니다. 공자학당 홍보 페이지는 jQuery·Swiper로 빠르게 납기했습니다.

2023.01 ~ 2023.04

### 무브인터렉티브 · 인턴/수습 · SI 개발

.NET 레거시 구조 분석 · DB 딕셔너리 (C# · ASP.NET · MSSQL · Doxygen)

- 문서화되지 않은 C# 결제 시스템을 Doxygen으로 분석해 클래스 구조와 API 명세를 추출하고 구조도로 정리했습니다.
- 약어 중심 테이블의 구조와 제약조건을 정리한 DB 딕셔너리를 만들고, 기존 결제 API 호출 흐름을 전수 조사해 외주 개발 범위를 좁히는 기획서 근거로 썼습니다.

2020.08 ~ 2021.06

### 한국뉴먼 · 사원 · IT 기획 (프리랜서)

스타트업 창업 지원 솔루션 기획·PM

- 창업 패키지 기획과 웹 테스트 문서 작성을 맡았고, 클라이언트와 개발자 사이에서 일정·우선순위를 조율했습니다.

## 학력

2014.03 ~ 2020.02

**한성대학교** · 행정학과 졸업 · 경제학과 부전공

## 경험 · 활동 · 교육

2026.03 ~ 2026.05

**휴먼IT교육센터** · AI 기반 서비스 개발 심화 과정

- Python 전처리·시각화, Scikit-Learn 앙상블(XGBoost·RandomForest), TensorFlow/Keras 시계열(CNN-LSTM).
- LangChain·LangGraph 상태 기반 대화 흐름, Neo4j 그래프 DB와 Chroma 벡터 DB를 결합한 하이브리드 RAG 실습.
- FastAPI로 모델 API를 만들고 Spring Boot 백엔드와 연동, Streamlit·Gradio UI, Docker·Kubernetes·Cloudtype 배포 실습.

**[팀 프로젝트] 오늘의 냉장고 — 식재료 관리·레시피 추천 (7인 팀, 2026.04 ~ 05)**

Spring Boot · Spring Security · Redis · Next.js · FastAPI · Gemini

- OAuth2 소셜 로그인과 JWT 인증, Redis 기반 Refresh 토큰 관리, @Async 메일 발송을 전담했습니다.
- 부족한 식재료를 네이버·11번가·쿠팡 API로 연결하면서 외부 장애 시 'Redis 캐시 → DB → 외부 API' 순서로 넘어가는 3단계 Fallback을 설계했습니다.
- Gemini로 영양 상태·쇼핑 맥락을 반영한 추천 문구를 생성하는 엔드포인트를 만들고, 응답 지연·오류 시 Fallback을 두었습니다.
- 백엔드·프론트 저장소가 분리된 7인 협업에서 .gitignore 규칙과 PR 리뷰를 정착시키고 프론트에 Vitest를 도입했습니다.

2022.08 ~ 2022.12

**티맥스티베로** · SW융합 개발자 과정 TABA 1기

- Tibero DB 실행 계획 분석, 백업 전략과 오류 해결 절차 정리, Linux Shell 기반 TTA BMT 시나리오 설계.
- ERD 설계·자료구조·머신러닝 기초를 매일 GitHub TIL과 Notion에 기록.

**[팀 프로젝트] ImageNet 이미지 분류 모델링 (팀, 2022.12 · TABA 공모전 최우수상)**

Python · PyTorch · TensorFlow/Keras · Flask · Azure · AWS

- ImageNet 50,000장을 로드해 레이블 인코딩과 데이터 증강을 수행하고, 커스텀 CNN으로 분류 정확도 0.8808을 기록했습니다.
- AlexNet·VGGNet·GoogLeNet·ResNet을 직접 구현해 비교하고 기울기 소실을 분석했습니다. Loss/Accuracy 곡선 비교로 단국대·CCCR 주관 공모전 최우수상.

2021.08 ~ 2022.02

**엔코아** · IoT·Bigdata·AI 기술융합 개발자 양성 과정 (960시간)

- 전자정부프레임워크·Java·MyBatis·MySQL 게시판 CRUD, Python 크롤링·머신러닝·CNN 기초.
- React · Spring Boot · FastAPI 기반 MVC 웹과 이미지 인식 모델 연동 팀 프로젝트.

**[팀 프로젝트] MIMO — 가상 메이크업 시뮬레이션 웹 (팀, 2022.01 ~ 03)**

React · Spring Boot · MySQL · TensorFlow (U-Net) · OpenCV · Google Cloud · Firebase

- U-Net Face Segmentation 모델(정확도 91.15%)을 연동해 웹캠 영상의 얼굴 영역 분리와 립 컬러 매핑을 구현했습니다.
- HTML/CSS 레이아웃을 React 컴포넌트로 설계해 프론트엔드를 전담했고, 장바구니·리뷰·구글 OAuth2 로그인을 Ajax로 백엔드와 연동했습니다.

## 자격

2021.01 GAIQ (Google Analytics Individual Qualification) · Google

2015.12 MOS Master 2013 · Microsoft

## 프로젝트

### 가이드롤 생산 모니터링 — 제조 지표 정의 재합의와 불변식 검증

2026.08 ~ 2026.09 회사 프로젝트 · 2026.10 공개 샘플 배포(개인) · 조회·분석 화면과 API·DB 구조, 배포

FastAPI · React/TypeScript · DuckDB · Parquet · Docker · Jenkins · pytest · Vitest

- 센서 노드 4대가 10분마다 측정한 속도·진동 사이클을 일간·주간·월간 화면으로 보여 주는 모니터링 시스템입니다.
- 한 주 통계에 '가동 1.2시간'과 '가동 0회'가 함께 떠서 원인을 코드 세 지점으로 좁혔습니다. 노드 카드·기간 통계·상태 분류가 서로 다른 가동 기준을 쓰고 있었습니다.
- '가동 1회'의 정의를 재합의안으로 올려 회의에서 확정하고 합산 함수를 하나로 모은 뒤, '0회면 0시간'·'주간 합 = 전체' 같은 불변식으로 36일치 화면을 재계산해 대조했습니다(위반 0건, 계약 테스트 26건).
- 퇴사 후 2026-10-02, 공개 샘플 데이터로 VPS에 예시포트폴리오를 배포하며 첫 조회 18.0초 → 0.89초 (DuckDB 연결 1회화 + Parquet 병합 사본, 전후 화면 데이터 8/8 동일)를 기록했습니다.

> 포트폴리오에는 공개 샘플 데이터와 샘플 화면만 썼고, 고객사명·운영 원자료는 싣지 않았습니다. 정지 중 진동으로 속도가 튀는 '유령 속도'는 가설 검증까지만 했고 임계값은 미정입니다.

Case Study ↗ 제조 × AI 사례 한 장 ↗

### KMovement 3.0 — 의미 검색 + 관계 기반 검색 여행 추천

2026.03 ~ · 개인 프로젝트 · AI 파이프라인 설계 · 백엔드/풀스택

FastAPI · Spring Boot · Next.js · PostgreSQL/PostGIS · ChromaDB (multilingual-e5-small) · Neo4j GraphRAG · Celery/Redis · MLflow · GCP

- 장소 설명의 의미 검색과 아티스트·장소 관계 기반 검색을 결합한 K-Culture 여행 추천 서비스입니다.
- 지역 같은 정확 조건, 검색 점수의 의미(QA 경로 distance ≤ 0.25), 근거가 없을 때의 응답을 경로별로 구분해 다룹니다.
- 영상·음성 생성처럼 무거운 작업은 Celery·Redis 큐로 API 응답과 분리했고, 사진 분석 결과에 따라 영상 생성·3D 패닝·뼈대 애니메이션 중 호출 모델을 나눕니다.

> 추천 정답성과 비용 개선은 별도 평가가 필요한 상태입니다. 지역 불일치 회귀 테스트와 무자료 응답 정책은 검증 뒤에 추가합니다.

Repository ↗ Live ↗

### SDUI — 서버 주도 UI 메타데이터 엔진 · Template Kit Studio 편집기

2026.01 ~ · 개인 프로젝트 · 코어 로직·아키텍처

Spring Boot · PostgreSQL · JWT · OAuth 2.0 · Next.js (App Router) · TypeScript · Vercel · AWS EC2 · GitHub Actions · Cloudflare Pages · Tauri

- ui_metadata 테이블의 화면 정보를 Spring Boot가 JSON UI 트리로 변환하고 Next.js DynamicEngine이 재귀 렌더링하는 구조를 구현했습니다. DB 행을 바꾸면 다음 UI 트리 요청에 반영되어 클라이언트 재배포 없이 화면 구성이 바뀝니다.
- Template Kit Studio: 그 화면 데이터(템플릿 manifest)를 개발자가 아니어도 브라우저에서 고치는 편집기입니다. CLI · Studio Web · Desktop(Tauri) · 게시된 정적 페이지가 같은 검증기와 플러그인 계약을 공유합니다.
- 사용자 등급별로 UI와 기능을 나누는 권한 제어를 넣었습니다.

> 현재 UI 트리 조회는 PostgreSQL을 직접 사용합니다. Redis TTL 캐시 코드는 호출 경로에 연결돼 있지 않아 성과로 적지 않습니다.

Repository ↗ Studio ↗

### Planning Harness · work-cycle — 승인 게이트가 있는 AI 기획 루프와 업무 사이클 도구

2026.06 ~ · 개인 프로젝트 · Cloudflare Workers · D1

Cloudflare Workers · Hono · D1 · Workers AI · Gemini · GitHub API · Claude Code 플러그인

- Planning Harness: 스킬 7개로 기획 산출물 생성 흐름을 고정하고, GitHub 이슈·프로젝트에 쓰는 작업은 dry-run과 사람 승인을 거칩니다. 승인 전 외부 작업 0건.
- work-cycle: 회의록 → AI 요약 → 검증 기록 → 칸반 → GitHub 이슈 → 회고의 하루 사이클. AI 요약이 검사에 걸리면 버려서 실패해도 흔적 0건이고, 요청 원문은 저장하지 않고 생성된 회의록만 D1에 남기며, 검증 표(항목·계산식·방법·결과·상태)는 사람이 적어야 그날이 닫힙니다.

> 공개 저장소는 회사 데이터를 뺀 사본입니다. 임베딩·벡터 검색은 쓰지 않습니다(회의 한 건 요약이라 검색 대상이 없음).

work-cycle 공개 데모 ↗ 설명 페이지 ↗ planning-harness-portfolio ↗ work-cycle-portfolio ↗

### Gomgom-AI — GPT 기반 심리 맞춤 음식 추천

2025.02 ~ 2025.06 · 개인 프로젝트 · 데이터 처리·백엔드

Python · Django · PostgreSQL · Redis · httpx · OpenAI API · Kakao Map API

- 외부 API 호출을 httpx 비동기로 바꿔 블로킹을 없애고, 반복되는 분석 결과와 식당 목록은 Redis에 캐싱했습니다.
- 위치와 감정 상태를 함께 보고 음식 카테고리를 제안하는 추천 흐름을 설계했습니다.

> AWS 운영은 종료했습니다. 속도 개선 수치는 측정 기록이 없어 적지 않습니다.

Repository ↗

# 자기소개서

## 데이터에서 개발로

행정학 학부에서 R과 SAS로 통계 데이터를 다루며 IT에 흥미를 느꼈습니다. 낯선 통계치 앞에서 논문과 질문으로 학습해 해당 과목 1등을 했고, 데이터를 어떻게 해석하느냐에 따라 결과가 달라진다는 것을 배웠습니다.

공무원 수험 생활에서는 방대한 규정의 예외와 조건을 정리하는 훈련을 했습니다. 이후 스타트업 창업 지원 회사에서 기획자·PM으로 창업 패키지 기획부터 웹 테스트 문서까지 맡으며, 클라이언트와 개발자 사이의 일정·우선순위 조율을 익혔습니다.

직접 만들어 보고 싶다는 생각이 커져 엔코아(Java·Spring), 티맥스(DB·인프라), 휴먼IT교육센터(LLM·RAG)를 거치며 개발자로 전환했습니다. 지금은 레거시 분석과 운영에서 배운 '왜 이 숫자가 나왔는지 설명하기'를 개발의 기본으로 삼고 있습니다.

## 7인 프로젝트에서 배운 협업

7명이 참여한 파이널 프로젝트 초반, 브랜치 병합 충돌과 소통 방식 차이로 갈등이 있었습니다. 저는 의견을 말하기 전에 Notion에 API 명세와 트러블슈팅 과정을 먼저 적고, Slack으로 에러 로그를 실시간 공유해 오해를 줄였습니다.

Jira로 스프린트와 우선순위를 가시화하고, Git PR 코드 리뷰와 .gitignore 규칙으로 환경 설정 충돌을 막았습니다. 팀원이 왜 그런 결정을 했는지 배경을 먼저 듣는 습관이 갈등을 푸는 데 가장 큰 도움이 됐습니다.

## AI 결과를 검증 가능한 업무 흐름 안에 두기

AI 도구로 구현 초안과 디버깅 후보를 빠르게 만들지만, 최종 요구사항과 임계값은 AI가 정하게 두지 않습니다. 프로젝트마다 AI가 만든 부분, 제가 결정한 기준, 실제 실행한 검증, 아직 확인하지 못한 부분을 구분해 적습니다.

제조 센서 대시보드에서는 화면에 숫자가 나와도 정의가 다르면 틀린 데이터라는 것을 배웠습니다. 그래서 가동의 정의를 한 장으로 먼저 고정하고, 같은 조건으로 횟수와 시간을 계산한 뒤 불변식 테스트로 검증했습니다.

AI 기획 루프와 회의록 도구에서는 밖으로 나가는 동작은 사람이 눌러야 실행되고, 검사에 걸린 결과는 버리고, 승인 뒤 내용이 바뀌면 승인을 취소하고, 사람의 기록으로만 완료되는 네 규칙을 지켰습니다. 제조 현장의 Agentic AI도 같은 원칙으로, 공정 판단 → 원인 추적 → 사람 승인 → 실행 → 검증 기록의 흐름으로 설계하고 있습니다.